



AI with Rav

Learn AI the way you actually think.



DAY 4 of 30

OPEN SOURCE CONTRIBUTION

Contributing a Test With No Bug Attached

WHAT YOU'LL LEARN TODAY

- > How to find functions with no tests
- > Which test cases are actually worth writing
- > Why "does not mutate the input" is the best test here
- > Why this took three weeks to merge



Contributing a Test With No Bug Attached

In One Line: I found a function in NLTK with no unit tests, wrote five, and they merged. No bug, no fix — and it still counts.

The gap in the advice

Days 1 to 3 were bug fixes. That is the highest-value contribution, and also the hardest to find — you need a real bug that nobody has claimed.

There is a second category that is far easier to find and still genuinely useful: **code that works but is not tested**.

Every mature project has some. It is not a sign of a bad project; it is what happens when a function written in 2008 keeps working and nobody has reason to revisit it.

How to find it

Open the test folder and the source folder side by side, and look for what is missing.

Find the gap in four seconds

nltk/util.py exports

```
everygrams
transitive_closure
invert_dict
pad_sequence
...more
```

test_util.py imports

```
from nltk.util import
    everygrams
```

*one function.
that is the whole file.*

Everything red is a contribution waiting to happen.

Comparing what the module exports against what the test file imports. The gap is your contribution.

In NLTK, **nltk/util.py** had a test file at **nltk/test/unit/test_util.py**. That file imported exactly one function:

```
from nltk.util import everygrams
```

One import. A module with many public functions. **transitive_closure** was one of the untested ones.



That single import line is the whole discovery method. If a test file imports one function from a module that exports fifteen, you have found fourteen candidates in about four seconds.

What the function does

transitive_closure takes a graph — who points at whom — and works out who can be reached from where, following the arrows as far as they go.

If **a** → **b** and **b** → **c**, then the closure says **a** reaches both **b** and **c**.

```
graph    = {"a": {"b"}, "b": {"c"}, "c": set()}
closure  = {"a": {"b", "c"}, "b": {"c"}, "c": set()}
```

Choosing the cases

This is where a useful test contribution separates from a lazy one. Do not write five tests of the same thing. Ask: **what would a future refactor plausibly break?**

Test	What it protects
Chain a→b→c	The core behaviour, following arrows more than one step
reflexive=True	The optional flag — each node includes itself
A cycle a↔b	That it terminates instead of looping forever
Empty graph	The boundary case, no crash on nothing
Does not mutate input	The one you would never notice breaking

The best test in the set

The last one is the one I would argue for hardest:

```
def test_transitive_closure_does_not_mutate_input():
    graph = {"a": {"b"}, "b": set()}
    snapshot = {k: set(v) for k, v in graph.items()}
    transitive_closure(graph)
    assert graph == snapshot
```

The function copies the sets internally, so the caller's graph is left alone. Nothing documents that. It is easy to lose in a refactor that adds an in-place update for speed.



And when it breaks, it does not raise. Your caller's data is quietly modified and something fails three functions later.

The cycle test earns its place for the same reason: transitive closure is a loop that keeps going until nothing changes. With a cycle, a naive implementation never stops. That test is the difference between a bug and a hung process.

Reading the implementation first

You cannot test what you do not understand, and **transitive_closure** has a detail that only shows up if you read it: it is a loop that runs **until nothing changes**.

```
# roughly what it does
while changed:
    changed = False
    for node, targets in closure.items():
        for t in list(targets):
            new = closure[t] - targets
            if new:
                targets |= new
                changed = True
```

Each pass adds any nodes reachable one more step out. When a pass adds nothing, it is done.

Once you see that shape, two of the five tests write themselves. A **cycle** is the case where "keep going until nothing changes" could mean "keep going forever" — so it must be tested. And because the loop mutates **targets**, whether the caller's sets get modified is a live question — so that must be tested too.

The test cases were not invented from a checklist. They came from reading the implementation and asking which of its assumptions could quietly stop being true.

Running them before opening the PR

Two commands, and both matter:

- Run the **new tests** and watch them pass
- Then run the **whole file** — **pytest nltk/test/unit/test_util.py** — to confirm you have not broken the tests that were already there

The second is not paranoia. Adding an import to a test file can shadow a name or trigger a slow import that changes another test's timing. It costs ten seconds to check.



A test-only PR that breaks existing tests is the worst possible first impression: you claimed to improve the test suite and made it red. Run the file, not just your function.

The honest framing

The PR description said what it was, with no inflation:

"noticed transitive_closure in nltk/util.py has no unit tests, so added a few to test_util.py next to the existing everygrams ones."

Not "improves code quality." Not "enhances the test suite." Just: this had no tests, now it has five, here is what they cover.

- Put them **next to the existing tests**, in the file that was already there
- Match the **existing style** — same imports, same naming, same plain **assert**
- State that **pre-commit passes** so the maintainer knows you ran the checks

It took three weeks

Opened 11 July. Merged 4 August.

That is normal, and it is not a rejection. A test-only PR is genuinely lower priority than a bug fix — nobody is blocked on it. It sits until a maintainer has a quiet afternoon.

Do not ping every few days. Do not close it and reopen it. A test PR that is correct and small will merge; your job after opening it is to be patient and to still be responsive when a comment finally arrives.

Being honest about the value

A test PR does not carry the same weight as a bug fix. If someone asks about your open source work and this is the strongest thing you have, it will not impress them.

What it does do:

- Gets you a **real merge** in a real project, with the full workflow — fork, branch, CI, review
- Teaches you a codebase from the **inside**, which is where your next bug fix comes from
- Is something you can find in an afternoon, when a good bug might take a week

Use it to learn the repo. Then find the bug.

What is next



The same move in a different repo — sentence-transformers — where the untested function was one whose *guard clause* was the whole point.

